

ENGINEER SANDBOX

Multi-Agent Software Engineering Workspace Simulation

Full Project Plan & Phased Build Guide • CLI + Python + Ollama + SQLite • Local
CPU-Optimised

CLI	Python 3.11+	Ollama / Mistral 7B	SQLite	Rich TUI	Async IO
-----	--------------	---------------------	--------	----------	----------

1. Project Overview

Engineer Sandbox is a locally-run, AI-powered CLI simulation in which the player inhabits a software engineering role and navigates realistic workplace scenarios. Each scenario features a cast of AI agents — bosses, senior developers, junior developers, product managers — who hold private conversations, call emergency meetings, send direct messages, and react dynamically to every player decision. A dedicated Scoring Agent evaluates performance at the end of each scenario, and a Scenario Generator uses those scores to craft progressively tailored challenges.

Core Design Principles

Fully Local	Runs entirely on a CPU laptop via Ollama. No cloud API keys, no latency bills, no internet required.
Emergent Realism	Agents hold parallel conversations simultaneously displayed in separate Rich panels, mimicking a real office where multiple discussions happen at once.
Adaptive Difficulty	The Scenario Generator tracks per-category performance scores and crafts new scenarios that target the player's documented weaknesses.
Memory-Per-Scenario	Agent memories reset at the start of each scenario, keeping context windows flat and inference fast while still feeling alive within a run.
Dedicated Scorer	A silent Scoring Agent reads the full event log after scenario completion and produces a structured, multi-dimensional performance report.

2. Technology Stack

Component	Choice	Reason
Model runtime	Ollama	One-command local LLM server; stable CPU support
LLM model	Mistral 7B Instruct	Best instruction-following at 7B size; ~4 GB RAM; 32k ctx
Language	Python 3.11+	asyncio, type hints, rich ecosystem
CLI framework	Typer + Rich	Clean commands + beautiful terminal panels and tables
Persistence	SQLite (sqlite3)	Zero-dependency, file-based, fast for this scale
Async I/O	asyncio + aiohttp	Non-blocking Ollama calls; sequential reality, async UX
Validation	Pydantic v2	Typed scenario/agent specs; catches bad LLM JSON early

Env management	uv / pip + venv	Fast dependency installs, reproducible environment
----------------	-----------------	--

CPU Optimisation Note

Ollama on CPU performs sequential inference regardless of how many requests arrive concurrently. The design embraces this: requests are issued asynchronously so Python never blocks, but the terminal displays each agent response as it streams in — making the system feel live rather than frozen. Limiting active agents per decision to 3–5 keeps wall-clock time per decision under 60 seconds on a mid-range laptop.

3. System Architecture

3.1 Agent Roster

Agent	Role	Memory Scope	Notes
Scenario Generator	Creates scenario + cast from player profile	Cross-scenario (reads SQLite scores)	Runs once per scenario start
Cast Agents (3–5)	Coworkers, boss, PM, juniors	Current scenario only — resets personalities	Personalities set by generator
Scoring Agent	Evaluates scenario outcome	None (reads full event log at end)	Silent observer; activates post-scenario

3.2 Core Game Loop

1	Scenario Generator reads player performance scores from SQLite, generates a new scenario spec (type, title, cast definitions, scheduled timeline, decision options).
2	Game Engine initialises Cast Agents with personality prompts and blank conversation histories.
3	Player reads the situation and makes a decision.
4	Each Cast Agent is queried sequentially via Ollama (async issued, sequential processed). Responses stream into separate Rich panels.
5	Agents may spawn new events: direct messages to player, side conversations with other agents, or emergency meeting calls. Each spawned event has a depth counter capped at 3.
6	Player responds to direct messages / meeting invites. Steps 3–5 repeat until a termination condition is met.
7	Scoring Agent reads the full event log and all conversation threads. Returns a structured JSON score report.
8	Post-scenario report is rendered in the terminal. Player may review any conversation thread.
9	Scores are written to SQLite. Generator is called for the next scenario.

3.3 Termination Conditions

- Deadline passed — the scenario's external deadline has expired.
- Resolution reached — all agents mark the situation as resolved.
- Critical failure — project collapses or a team member quits.
- Max turns — hard cap of 8 player decision turns per scenario.
- All agents resolved — every agent has had their final say.

3.4 Event Depth Limit

Every event carries a depth counter (0 = triggered directly by player decision). Child events increment this counter. No event may spawn a child if $\text{depth} \geq 3$. This prevents infinite reaction chains while preserving organic escalation.

4. Agent System Design

4.1 Agent State (per scenario)

Each Cast Agent is initialised at scenario start with the following fields. All fields reset when the scenario ends — there is no cross-scenario carry-over for Cast Agents.

<code>name / role</code>	String identity provided by generator
<code>personality_traits</code>	Dict of floats: directness, empathy, risk_aversion, political_savvy...
<code>values</code>	Ordered list: e.g. ['code_quality', 'team_harmony', 'shipping']
<code>goals_in_scenario</code>	What this agent wants to achieve this run
<code>constraints</code>	What limits their actions (budget frozen, can't override policy...)
<code>relationship_map</code>	Dict: {other_agent_name: {trust: 0–1, dynamic: str, history: []}}
<code>conversation_history</code>	Growing list of {role, content} turns passed to Ollama chat API
<code>emotional_state</code>	Current mood — updated after each interaction
<code>stress_level</code>	Float 0–1 — rises with emergencies, affects decision thresholds

4.2 Stateful Ollama via Chat API

Ollama is stateless between requests. Statefulness is achieved by maintaining each agent's `conversation_history` list in Python (and backed by SQLite within the scenario). Every call to Ollama's `/api/chat` endpoint passes the full messages array, giving the model implicit memory of all prior turns in that scenario.

```
POST http://localhost:11434/api/chat { "model": "mistral", "messages":
agent.conversation_history, "stream": false }
```

4.3 Communication Decision Logic

After each player decision, every Cast Agent independently evaluates whether to react. The assessment is a weighted combination of:

- `relevance_to_goals` — does this decision affect what I care about?
- `urgency` — is time-sensitive action needed?
- `relationship_trust` — am I comfortable reaching out to this target?
- `power_dynamic` — is this upward, lateral, or downward communication?
- `risk_of_speaking` — could my message backfire?

If the weighted score exceeds a threshold, the agent selects target(s): another agent, the player, or both. Conversations between agents are logged with a visibility flag (FULL / PARTIAL / SUMMARY / HIDDEN) that determines how much the player sees.

4.4 Parallel Conversation Display

Multiple conversations can be active simultaneously. Rich panels are rendered side-by-side (up to 2 columns) or stacked in a grid. The terminal updates in real time as each Ollama response arrives. Direct messages to the player are highlighted full-width and require a response before the simulation advances.

PARALLEL CONVERSATIONS

5. Scenario Generator

The Scenario Generator is the only agent with persistent, cross-scenario memory. It reads the player's performance profile from SQLite and produces a complete scenario specification including the cast, timeline, scheduled events, and decision options.

5.1 Scenario Types (initial set)

Type	Description
Conflict Resolution	Interpersonal friction, blame assignment, public vs private confrontation
Deadline Pressure	Time vs quality, scope negotiation, crunch decisions
Mentorship/Leadership	Junior feedback, delegation, psychological safety vs high standards
Strategic Decisions	Tech choices, architecture bets, build-vs-buy
Politics/Navigation	Managing up, org dynamics, career moves
Ethical Dilemmas	Corner-cutting, honesty under pressure, integrity tests
Team Dynamics	Morale, inclusion, belonging, remote vs in-person tension

5.2 Timeline Generation

The generator outputs two categories of events for each scenario:

- Scheduled Events — fixed deadlines, recurring meetings, external customer commitments. Each carries a flexibility score (0 = immovable, 1 = fully negotiable) and cascading consequences if missed.
- Emergent Events — spawned by agent reactions during play. Emergency meetings, urgent DMs, escalations to leadership. These compress or restructure the scheduled timeline.

5.3 Adaptive Generation Algorithm

Phase 1 — Weakness Detection	Identify scenario types with average score < 70. Identify scoring dimensions consistently low.
Phase 2 — Strength Leverage	Build scenarios that use the player's strong areas as scaffolding while stretching weak ones.
Phase 3 — Pattern Testing	If the player always avoids conflict, create a scenario where avoidance has explicit costs.
Phase 4 — Balanced Generation	Every 3–4 scenarios, include one the player is likely to succeed in (confidence maintenance).

Phase 5 — Narrative Arc

Early: foundational skills. Mid: political complexity. Late: integration of all areas.

6. Scoring System

Scoring is split between the Scoring Agent (objective dimensions) and individual Cast Agent satisfaction ratings (subjective relationship impact).

6.1 Scoring Dimensions

Dimension	Weight	What it measures
Immediate Outcome	20%	Did the presenting problem get resolved?
Team Health	20%	Morale, trust, and relationship state post-scenario
Long-term Impact	20%	Will this decision help or haunt the player later?
Learning Value	15%	Did the player grow or play it safe repeatedly?
Integrity	15%	Were decisions ethically sound and principled?
Agent Satisfaction	10%	Aggregate of each cast agent's 1–5 star rating

6.2 Post-Scenario Report Layout

The terminal report renders in this order:

- Scenario title, type, and duration
- Score bars per dimension (ASCII progress bars) + overall score
- Decision impact breakdown (pros/cons of key choices)
- Agent feedback with individual ratings and trust delta
- Relationship change table (before → after)
- Learning insights and detected decision patterns
- Key event timeline (chronological moments)
- Generator insight — which scenario type comes next and why
- Interactive menu: view full log | view timeline | review agent memories | new scenario | quit

7. Data Model (SQLite)

7.1 Persistent Tables (survive across scenarios)

Table	Key Columns	Purpose
player_performance	scenario_type, avg_score, last_5, patterns	Generator input — per-category scoring history
scenario_history	id, type, title, overall_score, timestamp	Archive of all completed scenarios
player_profile	role, reputation_json, decision_patterns	Player reputation visible to agents

7.2 Per-Scenario Tables (cleared at scenario end)

Table	Key Columns	Purpose
agent_conversations	agent_id, role, content, timestamp	Stateful Ollama history per agent
event_log	id, type, depth, initiator, participants	Every event that occurred
conversation_threads	id, participants, visibility, messages_json	Full parallel conversation threads
timeline_state	scheduled_events_json, emergent_events_json	Active timeline tracking
player_decisions	turn, decision_text, timestamp	Player's choices this scenario

8. Project Structure

```
engineer-sandbox/ ■■■ main.py # Entry point (Typer CLI) ■■■ game_loop.py # Core scenario
execution engine ■■■ requirements.txt ■ ■■■ agents/ ■ ■■■ scenario_generator.py # Reads
scores, generates spec ■ ■■■ cast_agent.py # Cast agent class + Ollama integration ■ ■■■
scoring_agent.py # Post-scenario evaluator ■ ■■■ agent_factory.py # Instantiates cast from
generator spec ■ ■■■ events/ ■ ■■■ event.py # Event dataclass + depth control ■ ■■■
timeline.py # Scheduled + emergent event manager ■ ■■■ orchestrator.py # Builds event
chains from agent responses ■ ■■■ storage/ ■ ■■■ database.py # All SQLite operations ■
■■■ schema.sql # Table definitions ■ ■■■ sandbox.db # Runtime database file ■ ■■■ ui/ ■
■■■ panels.py # Rich parallel conversation panels ■ ■■■ report.py # Post-scenario report
renderer ■ ■■■ timeline_view.py # Timeline sidebar display ■ ■■■ utils/ ■■■
ollama_client.py # Async Ollama wrapper ■■■ json_parser.py # Robust JSON extraction +
fallback ■■■ config.py # Model name, thresholds, constants
```

9. Phased Build Plan

Phase 1	Foundation & Ollama Integration	~1–2 days
Goal:	Get a working Python project that can talk to Ollama and persist data.	
→	Set up Python venv, install Typer, Rich, aiohttp, Pydantic, sqlite3	
→	Install Ollama and pull mistral:latest	
→	Write ollama_client.py — async wrapper for /api/chat and /api/generate	
→	Write database.py — schema creation and CRUD helpers	
→	Write config.py — model name, Ollama URL, threshold constants	
→	Smoke test: CLI command that sends a prompt to Ollama and prints response	
✓ Done when:	<i>python main.py runs, sends a prompt, gets a response, writes to SQLite.</i>	
Phase 2	Scenario Generator + Basic Cast	~2–3 days
Goal:	Generate a playable scenario with 2–3 agents and display it in the terminal.	
→	Write scenario_generator.py — prompt template that outputs scenario JSON	
→	Write Pydantic models for ScenarioSpec, AgentSpec, ScheduledEvent	
→	Write json_parser.py — extract JSON from messy LLM output with retry logic	
→	Write cast_agent.py — Agent class with conversation_history and decide() method	
→	Write agent_factory.py — instantiate Cast Agents from generator spec	
→	Write game_loop.py — present scenario text, collect player decision as string	
→	Display: basic Rich panel showing current situation and decision options	
✓ Done when:	<i>Player sees a scenario, types a decision, each agent responds sequentially in the terminal.</i>	

Phase 3	Parallel Conversation Display	~2–3 days
Goal:	Multiple agent conversations rendered simultaneously in separate Rich panels.	
→	Write panels.py — Rich Layout with parallel conversation columns	
→	Add conversation visibility logic (FULL / PARTIAL / SUMMARY / HIDDEN)	
→	Implement Live rendering — panels update as each Ollama response arrives	
→	Direct message panel — full-width, blocks game loop until player responds	
→	Add streaming output within each panel (token-by-token if Ollama supports)	
→	Wire agent communication targets — agent may DM player or talk to another agent	
✓ Done when:	Two or more conversations appear side-by-side, DMs require player response.	
Phase 4	Event System & Timeline	~2–3 days
Goal:	Agents can spawn emergent events; timeline tracks scheduled vs emergent.	
→	Write event.py — Event dataclass with type, depth, initiator, participants	
→	Write timeline.py — add/query scheduled and emergent events	
→	Write orchestrator.py — parse agent responses for event spawn signals	
→	Implement depth limit enforcement (max 3)	
→	Write timeline_view.py — sidebar panel showing TODAY / TOMORROW / DEADLINE	
→	Termination condition checker — evaluate after each player turn	
→	Emergency meeting flow — agent calls meeting, player accepts/defers/counters	
✓ Done when:	Agent triggers emergency meeting; timeline panel updates; scenario ends on deadline.	

Phase 5	Scoring Agent + Post-Scenario Report	~2 days
Goal:	Full evaluation pipeline and rich post-scenario analytics display.	
→	Write scoring_agent.py — reads full event log, all threads, returns score JSON	
→	Collect agent satisfaction ratings (each cast agent rates 1–5 at scenario end)	
→	Write report.py — render all scoring dimensions with ASCII progress bars	
→	Render relationship change table (trust before → after)	
→	Render key event timeline (critical moments)	
→	Implement conversation log browser — filter by agent, impact, timestamp	
→	Write scores back to player_performance SQLite table	
✓ Done when:	Full report renders after scenario end; player can browse any conversation thread.	
Phase 6	Adaptive Generator + Player Profile	~2–3 days
Goal:	Generator reads historical scores and adapts. Player has a reputation profile.	
→	Build performance profile query — per-type averages, trends, decision patterns	
→	Add player_profile table — reputation scores agents reference in prompts	
→	Update generator prompt to include performance profile context	
→	Implement Phases 1–5 of the adaptive algorithm (weakness detection → narrative arc)	
→	Prevent generator from repeating same scenario type back-to-back	
→	Generator insight section in post-scenario report	
→	Validate adaptation over 5+ scenarios end-to-end	
✓ Done when:	After 5 scenarios, generator demonstrably targets player's weakest category.	

Phase 7	Polish, Testing & Edge Cases	~2 days
Goal:	Production-quality stability, good error handling, smooth UX.	
→	JSON parse failure handling — retry with stricter prompt, fallback defaults	
→	Ollama timeout handling — 120s timeout, graceful degradation	
→	Database migration safety — schema versioning	
→	CLI help text and onboarding (first-run instructions)	
→	Config file support — user can change model, thresholds	
→	Full end-to-end playthrough test (3+ scenarios)	
→	README with setup instructions (ollama install, model pull, python env)	
✓ Done when:	<i>Project runs clean start-to-finish with no crashes on bad LLM output.</i>	

10. MVP Scope vs. Future Enhancements

MVP (Phases 1–5)	Post-MVP (Phase 6+)
✓ 3 scenario types	→ All 7+ scenario types
✓ 3 cast agents per scenario	→ Up to 6 agents
✓ Sequential Ollama calls (streamed)	→ Batched Ollama if supported
✓ Parallel panel display (Rich)	→ Full split-screen TUI
✓ Basic scoring (5 dimensions)	→ Scoring with agent satisfaction weights
✓ Conversation log browser	→ Export to file / shareable replay
✓ Fixed scenario termination rules	→ Dynamic termination via agents
✓ Player decides in freeform text	→ Structured decision menus
✓ Scenario generator (basic)	→ Fully adaptive generator (all 5 phases)
✓ SQLite persistence	→ Optional JSON export

Estimated Total Build Time



Total estimated: ~18 developer-days (solo) or ~2 weeks part-time.